DocConnect — First Review Guide & Script

Speaker Role: Roshell Miranda (P1 — Full Stack & Git Repository Owner)
Focus Slides: 4, 5, and 6 | Comprehensive Q&A & Slide-by-Slide Knowledge

---

Part 1: Your Spoken Presentation Script (Slides 4 – 6)

Speak calmly at a steady pace. Use the exact words below for each slide.

■■ Slide 4: Scope & Expected Outcomes

Spoken Script:
"Good morning / afternoon respected panel members.

Moving to the **Scope and Expected Outcomes** of DocConnect:

On the **Scope** side:
• **Single Shared Repository:** We structured our codebase into four parallel modules: full-stack application, Docker containerization, Terraform cloud infrastructure, and Kubernetes orchestration.
• **Cloud Integration:** All four modules integrate into a single unified deployment on AWS Kubernetes.
• **Synthetic Demo Data:** We use realistic synthetic healthcare records so we can test booking workflows without privacy or compliance risks.
• **End-to-End Pipeline:** Our scope spans code commit → automated testing → container build → cloud deployment → horizontal autoscaling. Heavy clinical compliance like HIPAA is kept out of scope for this mini-project.

On the **Expected Outcomes** side:
1. A live, responsive **DocConnect appointment portal** reachable end-to-end.
2. **Zero-touch CI/CD automation** via Jenkins triggering builds and tests on every commit.
3. **Visible Horizontal Pod Autoscaling (HPA)** spinning up extra backend replicas during simulated traffic spikes.
4. **Automatic self-healing**, where Kubernetes automatically detects and restarts failed pods in seconds."

■■ Slide 5: SDG Mapping & Societal Impact

Spoken Script:
"Next, I will explain how DocConnect directly contributes to the **UN Sustainable Development Goals (SDGs)** and supports community healthcare:

• **SDG 3 — Good Health & Well-Being (Primary Goal):** Small clinics and rural centers often suffer from crashed servers and lost appointments. DocConnect ensures uninterrupted healthcare access and continuity of care through zero-downtime deployment.
• **SDG 9 — Industry, Innovation & Infrastructure:** We provide a resilient, automated cloud infrastructure with CI/CD and self-healing, giving community healthcare providers enterprise-level reliability without requiring in-house DevOps staff.
• **SDG 12 — Responsible Consumption & Production:** Instead of paying for oversized, always-on servers, Kubernetes autoscaling dynamically scales compute pods up during peak morning hours and scales down during quiet hours, reducing cloud energy waste and operational costs.

Societal Impact: DocConnect is ideal for small clinics, NGO medical camps, and telehealth setups to prevent double-bookings, survive seasonal traffic spikes (like flu season), and run with zero maintenance overhead."

■■ Slide 6: Literature Review & Research Gap

Spoken Script:
"Now looking at the **Literature Review**:

We analyzed recent research papers in cloud-native healthcare and DevOps adoption:
1. **Cloud-native Healthcare (Salunkhe et al.):** Highlighted that Docker and Kubernetes improve deployment consistency and high availability.
• *Research Gap:* Existing studies are mostly conceptual and lack an end-to-end practical DevOps lifecycle.
2. **DevOps in Telehealth (Vemuri & Oluwaseyi):** Demonstrated that CI/CD and Infrastructure-as-Code eliminate deployment bottlenecks.
• *Research Gap:* Very few practical implementations connect real appointment management systems with automated pipelines.
3. **Microservices Scalability (Akerele et al.):** Proved that modular microservices offer superior resource efficiency compared to monoliths.
• *Research Gap:* Prior work focuses on architectural diagrams without automated infrastructure provisioning.

How DocConnect Fills the Gap: We deliver both a working full-stack appointment application and an automated, reproducible cloud pipeline using Jenkins, Terraform, and AWS Kubernetes.

(Handover line): Now, I would like to hand over to my teammate to present our Proposed System architecture."

Part 2: Complete Slide-by-Slide Crash Course (Slides 1 to 12)

Know the entire deck so you can comfortably handle any question across the presentation.

Slide	Slide Title	Summary & Key Takeaway (In Plain English)
1	Title Slide	**DocConnect: Healthcare Appointment System with DevOps Automation**. Team of 4 members: Simone, Shraddha, Roshell (You), Kalanidhi.
2	Problem Statement	Small clinics deploy websites manually. When servers fail or traffic spikes during flu season, appointments break and recovery is slow due to lack of in-house DevOps staff.
3	Objectives	5 goals: 1) Full-stack app (React, Node, Postgres), 2) Docker containers, 3) Jenkins CI/CD, 4) Terraform AWS IaC, 5) Kubernetes auto-scaling & self-healing.
4	Scope & Expected Outcomes *(Your Slide)*	Shared repo, 4 workstreams, synthetic data, AWS deployment. Proves zero-touch CI/CD, HPA autoscaling, and pod self-healing.
5	SDG & Societal Impact *(Your Slide)*	SDG 3 (Healthcare uptime), SDG 9 (Resilient infrastructure), SDG 12 (Cost & energy efficiency via HPA). Benefits small clinics & NGO camps.
6	Literature Review *(Your Slide)*	Surveyed 4 papers. Identifies gap: existing research is theoretical; DocConnect provides the missing end-to-end practical cloud implementation.
7	Proposed System	System architecture: Patient/Clinic → React → Express REST API → PostgreSQL RDS, managed with Jenkins, Docker, and AWS Kubernetes.
8	DevOps Toolchain	The 6 tools: **Git/GitHub** (Source), **Jenkins** (CI/Build), **Docker** (Containers), **Terraform** (IaC), **AWS** (Cloud), **Kubernetes** (Orchestration).
9	Roadmap	8-week execution plan dividing App, Docker/Jenkins, AWS/Terraform, and Kubernetes with progressive milestone checkpoints.
10	Group Organization	Role division: **P1 Roshell** (Full Stack & Git), **P2 Kalanidhi** (Docker/Jenkins), **P3 Shraddha** (AWS/Terraform), **P4 Simone** (Kubernetes).
11-12	References & Thank You	Academic citations for literature review, IEEE/SSRN papers, and closing slide.

Part 3: Full Stack Developer Mastery (Your Role — P1)

Since you built the entire application, here are your core implementation details:

1. React 19 Frontend (Vite): • Responsive booking interface, dynamic calendar/slot picker, appointment management table, cancellation confirmation modal. • Modular architecture: components/, pages/, services/api.js.

2. Node.js & Express REST API: • GET /health: Health probe verifying live PostgreSQL connectivity (used by Kubernetes liveness/readiness probes). • GET /appointments: Lists booked appointments with filtering. • POST /appointments: Schedules a new visit with input validation middleware. • PUT /appointments/:id & DELETE /appointments/:id: Update or cancel appointment status.

3. PostgreSQL Database Layer: • pg.Pool connection pooling to handle concurrent client requests smoothly. • **Double-Booking Prevention:** Composite unique constraints on (doctor_id, appointment_date, time_slot). • **Security:** Fully parameterized SQL queries ($1, $2...) preventing SQL Injection.

Part 4: Panel Q&A & Viva Defense Cheat Sheet

Anticipated questions from the review panel and how to answer them confidently:

Q1: Why did you use synthetic data instead of real patient data (Slide 4)?
Answer: Real clinical data requires strict HIPAA/GDPR compliance, data encryption at rest, and audit tracking, which is out of scope for this mini-project. Synthetic data lets us thoroughly test high-traffic booking, double-booking logic, and Kubernetes autoscaling safely.

Q2: What is the main research gap you identified in your Literature Review (Slide 6)?
Answer: Existing academic papers discuss cloud-native healthcare or microservices conceptually, but lack practical, reproducible implementations. DocConnect bridges this gap by combining a complete full-stack appointment application with an end-to-end automated Jenkins CI/CD and AWS Kubernetes pipeline.

Q3: How does your backend integrate with Kubernetes (Slide 7 & P4's work)?
Answer: I created a dedicated /health endpoint in Express. When deployed on Kubernetes, K8s periodically pings this endpoint as **Liveness and Readiness probes**. If database connectivity drops or Express hangs, Kubernetes detects it and automatically restarts the pod (self-healing).

Q4: How do you prevent double-booking of doctors in your application?
Answer: Double-booking is prevented at two layers: 1) At the database layer, PostgreSQL enforces a composite unique constraint on (doctor_id, appointment_date, time_slot). 2) In Express backend, we check slot availability and return a 409 Conflict status code if already booked.

Q5: Why did you select PostgreSQL instead of MongoDB?
Answer: Healthcare appointment booking requires strict **ACID compliance** and relational data integrity between doctors, patients, and time slots. PostgreSQL guarantees that two concurrent booking requests for the exact same slot will never collide.

Q6: How does Kubernetes Horizontal Pod Autoscaling (HPA) work for your application?
Answer: Our Express backend is completely stateless inside Docker containers. Kubernetes monitors CPU and memory usage via the Metrics Server. When appointment traffic surges, HPA scales backend pods from 2 to 5+ replicas automatically, distributing the load.

Q7: What is the purpose of Terraform in your project?
Answer: Terraform is our **Infrastructure as Code (IaC)** tool. Instead of manually clicking buttons in the AWS console, Terraform provisions our AWS VPC networking, subnets, IAM roles, and EKS Kubernetes cluster in a repeatable, version-controlled script.

Part 5: Quick Tips for Presentation Day

- **Pacing & Eye Contact:** Speak clearly at an easy pace. Slides 4 to 6 should take about **2 to 2.5 minutes**.
- **Ownership:** Show confidence because you wrote the entire full-stack application code.
- **Teamwork Handover:** When finishing Slide 6, smoothly pass the baton: *'Now, I would like to hand over to [Teammate Name] to walk us through the Proposed System.'*